DL_04_entrenamiento

March 13, 2026

1 Entrenamiento de Modelos Deep Learning

1.1 Contenido

1. Optimizadores
2. Learning Rate Scheduling
3. Regularización
4. Data Augmentation
5. Early Stopping y Checkpointing
6. Debugging y Diagnóstico
7. Ciclo de Entrenamiento Completo

---

[1]:
```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import numpy as np
```

1.2 1. Optimizadores

Los optimizadores actualizan los parámetros del modelo para minimizar la pérdida.

1.2.1 Gradient Descent Básico

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

1.2.2 Comparativa de Optimizadores

[2]:
```
# Comparación de optimizadores
modelo = nn.Linear(10, 2)

optimizadores = {
    'SGD': optim.SGD(modelo.parameters(), lr=0.01),
    'SGD + Momentum': optim.SGD(modelo.parameters(), lr=0.01, momentum=0.9),
    'Adam': optim.Adam(modelo.parameters(), lr=0.001),
    'AdamW': optim.AdamW(modelo.parameters(), lr=0.001, weight_decay=0.01),
```

```
    'RMSprop': optim.RMSprop(modelo.parameters(), lr=0.001)
}

print("=== OPTIMIZADORES ===")
print(f"{'Nombre':<20} {'Descripción'}")
print("-" * 60)
print(f"{'SGD':<20} Básico, puede ser lento")
print(f"{'SGD + Momentum':<20} Acelera convergencia, suaviza oscilaciones")
print(f"{'Adam':<20} Adaptativo, buen default")
print(f"{'AdamW':<20} Adam con weight decay correcto")
print(f"{'RMSprop':<20} Adaptativo, bueno para RNNs")
```

```
=== OPTIMIZADORES ===
Nombre               Descripción
------------------------------------------------------------
SGD                  Básico, puede ser lento
SGD + Momentum       Acelera convergencia, suaviza oscilaciones
Adam                 Adaptativo, buen default
AdamW                Adam con weight decay correcto
RMSprop              Adaptativo, bueno para RNNs
```

1.2.3 Recomendaciones:

Caso de uso	Optimizador recomendado
Default general	Adam o AdamW
Visión (CNNs)	SGD + Momentum (0.9)
NLP / Transformers	AdamW
Fine-tuning	Adam con lr bajo (1e-5)
RNNs	Adam o RMSprop

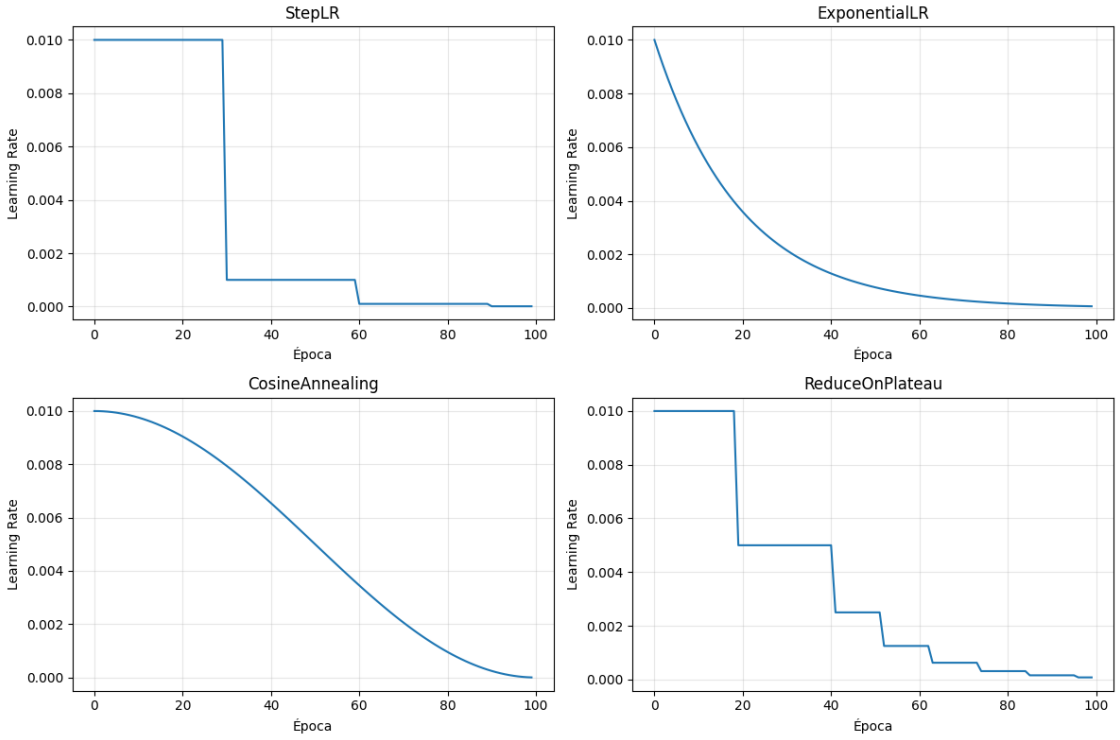
1.3 2. Learning Rate Scheduling

El learning rate es el hiperparámetro más importante. Ajustarlo durante el entrenamiento mejora la convergencia.

[3]:
```
# Visualizar diferentes schedulers
modelo = nn.Linear(10, 2)
epochs = 100

schedulers = {
    'StepLR': lambda opt: optim.lr_scheduler.StepLR(opt, step_size=30, gamma=0.
 ↪1),
    'ExponentialLR': lambda opt: optim.lr_scheduler.ExponentialLR(opt, gamma=0.
 ↪95),
    'CosineAnnealing': lambda opt: optim.lr_scheduler.CosineAnnealingLR(opt,␣
 ↪T_max=epochs),
```

```
    'ReduceOnPlateau': lambda opt: optim.lr_scheduler.ReduceLROnPlateau(opt,␣
  ↪factor=0.5, patience=10),
}

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes = axes.flatten()

for idx, (name, scheduler_fn) in enumerate(schedulers.items()):
    optimizer = optim.Adam(modelo.parameters(), lr=0.01)
    scheduler = scheduler_fn(optimizer)

    lrs = []
    for epoch in range(epochs):
        lrs.append(optimizer.param_groups[0]['lr'])
        if name == 'ReduceOnPlateau':
            scheduler.step(np.random.random())  # Simular val_loss
        else:
            scheduler.step()

    axes[idx].plot(lrs)
    axes[idx].set_title(name)
    axes[idx].set_xlabel('Época')
    axes[idx].set_ylabel('Learning Rate')
    axes[idx].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

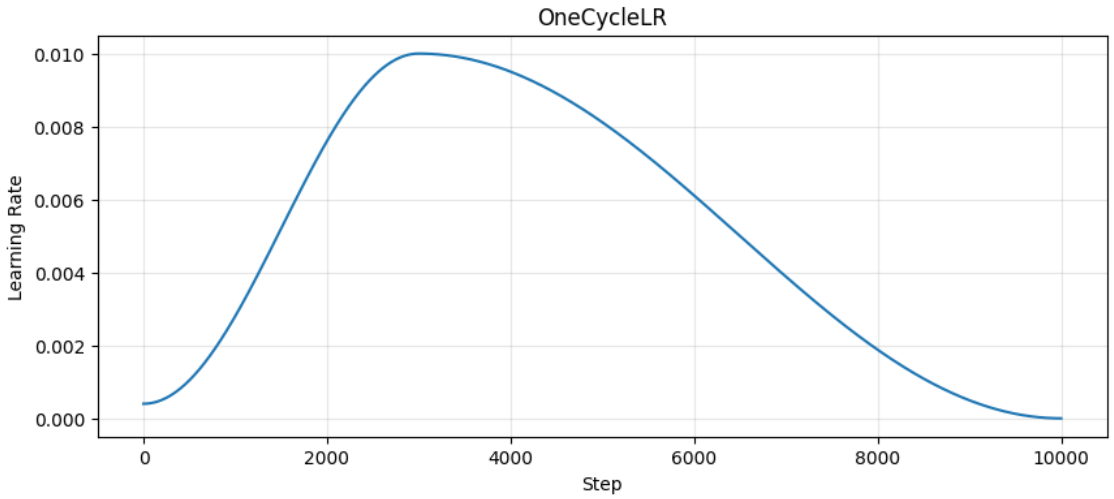
```
/tmp/ipykernel_32653/3043802872.py:25: UserWarning: Detected call of
`lr_scheduler.step()` before `optimizer.step()`. In PyTorch 1.1.0 and later, you
should call them in the opposite order: `optimizer.step()` before
`lr_scheduler.step()`.  Failure to do this will result in PyTorch skipping the
first value of the learning rate schedule. See more details at
https://pytorch.org/docs/stable/optim.html#how-to-adjust-learning-rate
  scheduler.step()
```



[4]:
```python
# OneCycleLR - muy efectivo para convergencia rápida
modelo = nn.Linear(10, 2)
optimizer = optim.Adam(modelo.parameters(), lr=0.001)

scheduler = optim.lr_scheduler.OneCycleLR(
    optimizer,
    max_lr=0.01,           # LR máximo
    epochs=100,
    steps_per_epoch=100,   # Batches por época
    pct_start=0.3,         # 30% de épocas subiendo
    anneal_strategy='cos'  # Decaimiento coseno
)

lrs = []
for _ in range(100 * 100):  # epochs * steps
    lrs.append(optimizer.param_groups[0]['lr'])
    scheduler.step()

plt.figure(figsize=(10, 4))
plt.plot(lrs)
plt.title('OneCycleLR')
plt.xlabel('Step')
plt.ylabel('Learning Rate')
```

```
plt.grid(True, alpha=0.3)
plt.show()
```

```
/tmp/ipykernel_32653/2476031818.py:17: UserWarning: Detected call of
`lr_scheduler.step()` before `optimizer.step()`. In PyTorch 1.1.0 and later, you
should call them in the opposite order: `optimizer.step()` before
`lr_scheduler.step()`.  Failure to do this will result in PyTorch skipping the
first value of the learning rate schedule. See more details at
https://pytorch.org/docs/stable/optim.html#how-to-adjust-learning-rate
  scheduler.step()
```



1.4 3. Regularización

Técnicas para prevenir overfitting.

[5]:
```
class ModeloRegularizado(nn.Module):
    """Ejemplo con todas las técnicas de regularización."""

    def __init__(self):
        super().__init__()

        self.features = nn.Sequential(
            # Capa 1
            nn.Linear(784, 256),
            nn.BatchNorm1d(256),      # 1. Batch Normalization
            nn.ReLU(),
            nn.Dropout(0.3),           # 2. Dropout

            # Capa 2
            nn.Linear(256, 128),
```

```
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Dropout(0.3),

            # Capa de salida
            nn.Linear(128, 10)
        )

    def forward(self, x):
        return self.features(x.view(x.size(0), -1))

modelo = ModeloRegularizado()

# 3. Weight Decay (L2 regularization) en el optimizador
optimizer = optim.Adam(modelo.parameters(), lr=0.001, weight_decay=1e-4)

print("Técnicas de regularización:")
print("1. BatchNorm: Normaliza activaciones, acelera entrenamiento")
print("2. Dropout: Desactiva neuronas aleatoriamente (30%)")
print("3. Weight Decay: Penaliza pesos grandes (L2)")
```

```
Técnicas de regularización:
1. BatchNorm: Normaliza activaciones, acelera entrenamiento
2. Dropout: Desactiva neuronas aleatoriamente (30%)
3. Weight Decay: Penaliza pesos grandes (L2)
```

[6]:
```
# Label Smoothing
print("=== LABEL SMOOTHING ===")

# Sin label smoothing: target = [0, 0, 1, 0, 0]
# Con label smoothing (0.1): target = [0.02, 0.02, 0.92, 0.02, 0.02]

loss_fn = nn.CrossEntropyLoss(label_smoothing=0.1)
print("CrossEntropyLoss con label_smoothing=0.1")
print("Suaviza los targets, previene sobreconfianza")
```

```
=== LABEL SMOOTHING ===
CrossEntropyLoss con label_smoothing=0.1
Suaviza los targets, previene sobreconfianza
```

1.5 4. Data Augmentation

Genera variaciones de los datos de entrenamiento para mejorar generalización.

[7]:
```
from torchvision import transforms

# Augmentation para imágenes
transform_train = transforms.Compose([
```

```
    transforms.RandomHorizontalFlip(p=0.5),      # Voltear horizontalmente
    transforms.RandomRotation(15),                # Rotar ±15°
    transforms.RandomCrop(32, padding=4),         # Recortar con padding
    transforms.ColorJitter(                       # Variaciones de color
        brightness=0.2,
        contrast=0.2,
        saturation=0.2,
        hue=0.1
    ),
    transforms.RandomAffine(                      # Transformaciones afines
        degrees=0,
        translate=(0.1, 0.1),
        scale=(0.9, 1.1)
    ),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
    transforms.RandomErasing(p=0.2)               # Borrar parches aleatorios
])

# Para validación/test: SIN augmentation
transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

print("Data Augmentation aplicado solo en entrenamiento")
```

```
Data Augmentation aplicado solo en entrenamiento
```

[8]:
```
# Visualizar augmentation
from PIL import Image

# Crear imagen de ejemplo
img = Image.new('RGB', (32, 32), color='blue')

# Simular diferentes augmentations
augmentations = [
    ('Original', transforms.ToTensor()),
    ('HFlip', transforms.Compose([transforms.RandomHorizontalFlip(p=1),␣
 ↪transforms.ToTensor()])),
    ('Rotation', transforms.Compose([transforms.RandomRotation(30), transforms.
 ↪ToTensor()])),
    ('ColorJitter', transforms.Compose([transforms.ColorJitter(0.5, 0.5, 0.5, 0.
 ↪2), transforms.ToTensor()]))
]

print("Tipos de Data Augmentation:")
```

```
for name, _ in augmentations:
    print(f"  - {name}")
```

```
Tipos de Data Augmentation:
  - Original
  - HFlip
  - Rotation
  - ColorJitter
```

1.6 5. Early Stopping y Checkpointing

[9]:
```
import copy

class EarlyStopping:
    """
    Detiene el entrenamiento si no hay mejora en val_loss.
    """
    def __init__(self, patience=7, min_delta=0.0001):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_loss = None
        self.best_weights = None
        self.should_stop = False

    def __call__(self, val_loss, model):
        if self.best_loss is None:
            self.best_loss = val_loss
            self.best_weights = copy.deepcopy(model.state_dict())
        elif val_loss > self.best_loss - self.min_delta:
            self.counter += 1
            if self.counter >= self.patience:
                self.should_stop = True
                # Restaurar mejores pesos
                model.load_state_dict(self.best_weights)
        else:
            self.best_loss = val_loss
            self.best_weights = copy.deepcopy(model.state_dict())
            self.counter = 0

        return self.should_stop

# Uso
early_stopping = EarlyStopping(patience=5)
print("EarlyStopping configurado con patience=5")
```

```
EarlyStopping configurado con patience=5
```

[10]:
```python
# Checkpointing
def guardar_checkpoint(modelo, optimizador, epoca, loss, ruta):
    """Guarda checkpoint completo."""
    torch.save({
        'epoch': epoca,
        'model_state_dict': modelo.state_dict(),
        'optimizer_state_dict': optimizador.state_dict(),
        'loss': loss
    }, ruta)
    print(f"Checkpoint guardado: {ruta}")

def cargar_checkpoint(modelo, optimizador, ruta):
    """Carga checkpoint y retorna época."""
    checkpoint = torch.load(ruta)
    modelo.load_state_dict(checkpoint['model_state_dict'])
    optimizador.load_state_dict(checkpoint['optimizer_state_dict'])
    return checkpoint['epoch'], checkpoint['loss']

print("Funciones de checkpointing definidas")
```

```
Funciones de checkpointing definidas
```

1.7 6. Debugging y Diagnóstico

[11]:
```python
# Verificar gradientes
def verificar_gradientes(modelo):
    """Detecta gradientes problemáticos."""
    print("=== ANÁLISIS DE GRADIENTES ===")
    for name, param in modelo.named_parameters():
        if param.grad is not None:
            grad = param.grad
            print(f"{name}:")
            print(f"  Mean: {grad.mean().item():.6f}")
            print(f"  Std:  {grad.std().item():.6f}")
            print(f"  Max:  {grad.max().item():.6f}")
            print(f"  Min:  {grad.min().item():.6f}")

            # Detectar problemas
            if grad.isnan().any():
                print("    CONTIENE NaN!")
            if grad.isinf().any():
                print("    CONTIENE Inf!")
            if (grad.abs() < 1e-7).all():
                print("    VANISHING GRADIENT!")

# Ejemplo
modelo = nn.Linear(10, 2)
```

```
x = torch.randn(32, 10)
y = torch.randint(0, 2, (32,))

output = modelo(x)
loss = nn.CrossEntropyLoss()(output, y)
loss.backward()

verificar_gradientes(modelo)
```

```
=== ANÁLISIS DE GRADIENTES ===
weight:
  Mean: -0.000000
  Std:  0.177618
  Max:  0.340494
  Min:  -0.340494
bias:
  Mean: -0.000000
  Std:  0.013388
  Max:  0.009467
  Min:  -0.009467
```

[12]:
```
# Curvas de aprendizaje
def plot_curvas_aprendizaje(historial):
    """Visualiza curvas de entrenamiento."""
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    epochs = range(1, len(historial['train_loss']) + 1)

    # Loss
    axes[0].plot(epochs, historial['train_loss'], 'b-', label='Train')
    axes[0].plot(epochs, historial['val_loss'], 'r-', label='Val')
    axes[0].set_title('Loss')
    axes[0].set_xlabel('Época')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    # Accuracy
    axes[1].plot(epochs, historial['train_acc'], 'b-', label='Train')
    axes[1].plot(epochs, historial['val_acc'], 'r-', label='Val')
    axes[1].set_title('Accuracy')
    axes[1].set_xlabel('Época')
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()
```

```
    # Diagnóstico
    gap = historial['train_acc'][-1] - historial['val_acc'][-1]
    print(f"\nDiagnóstico:")
    print(f"  Train acc: {historial['train_acc'][-1]:.2f}%")
    print(f"  Val acc: {historial['val_acc'][-1]:.2f}%")
    print(f"  Gap: {gap:.2f}%")

    if gap > 10:
        print("    OVERFITTING: Gran diferencia train/val")
    elif historial['train_acc'][-1] < 70:
        print("    UNDERFITTING: Accuracy muy bajo")
    else:
        print("    Modelo balanceado")
```

1.8 7. Ciclo de Entrenamiento Completo

[13]:
```
def entrenar_completo(
    modelo, train_loader, val_loader,
    epochs=100, lr=0.001, device='cpu',
    patience=10
):
    """
    Ciclo de entrenamiento completo con todas las mejores prácticas.
    """
    modelo = modelo.to(device)

    # Configuración
    loss_fn = nn.CrossEntropyLoss()
    optimizer = optim.AdamW(modelo.parameters(), lr=lr, weight_decay=0.01)
    scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)
    early_stopping = EarlyStopping(patience=patience)

    # Historial
    historial = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc':␣
 ↪[]}

    for epoch in range(epochs):
        # ====== ENTRENAMIENTO ======
        modelo.train()
        train_loss, train_correct, train_total = 0, 0, 0

        for batch_x, batch_y in train_loader:
            batch_x, batch_y = batch_x.to(device), batch_y.to(device)

            optimizer.zero_grad()
            output = modelo(batch_x)
            loss = loss_fn(output, batch_y)
```

```
            loss.backward()

            # Gradient clipping
            torch.nn.utils.clip_grad_norm_(modelo.parameters(), max_norm=1.0)

            optimizer.step()

            train_loss += loss.item() * batch_x.size(0)
            train_correct += (output.argmax(1) == batch_y).sum().item()
            train_total += batch_y.size(0)

        # ====== VALIDACIÓN ======
        modelo.eval()
        val_loss, val_correct, val_total = 0, 0, 0

        with torch.no_grad():
            for batch_x, batch_y in val_loader:
                batch_x, batch_y = batch_x.to(device), batch_y.to(device)
                output = modelo(batch_x)
                loss = loss_fn(output, batch_y)

                val_loss += loss.item() * batch_x.size(0)
                val_correct += (output.argmax(1) == batch_y).sum().item()
                val_total += batch_y.size(0)

        # Calcular métricas
        train_loss /= train_total
        val_loss /= val_total
        train_acc = 100 * train_correct / train_total
        val_acc = 100 * val_correct / val_total

        historial['train_loss'].append(train_loss)
        historial['val_loss'].append(val_loss)
        historial['train_acc'].append(train_acc)
        historial['val_acc'].append(val_acc)

        # Scheduler step
        scheduler.step()

        # Logging
        if (epoch + 1) % 5 == 0:
            print(f"Epoch {epoch+1}/{epochs}")
            print(f"  Train Loss: {train_loss:.4f}, Acc: {train_acc:.2f}%")
            print(f"  Val Loss: {val_loss:.4f}, Acc: {val_acc:.2f}%")

        # Early stopping
        if early_stopping(val_loss, modelo):
```

```
            print(f"\nEarly stopping en época {epoch+1}")
            break

    return historial

print("Función de entrenamiento completo definida")
```

```
Función de entrenamiento completo definida
```

1.9 Checklist de Entrenamiento

1.9.1 Antes de entrenar:

- ☐ Verificar shapes de datos
- ☐ Normalizar datos
- ☐ Split train/val/test
- ☐ Establecer semilla para reproducibilidad

1.9.2 Durante entrenamiento:

- ☐ Monitorear loss train Y val
- ☐ Usar early stopping
- ☐ Guardar checkpoints
- ☐ Verificar gradientes (NaN, vanishing)

1.9.3 Si hay overfitting:

- ☐ Aumentar dropout
- ☐ Más data augmentation
- ☐ Reducir modelo (menos capas/neuronas)
- ☐ Aumentar weight decay

1.9.4 Si hay underfitting:

- ☐ Aumentar capacidad del modelo
- ☐ Reducir regularización
- ☐ Entrenar más épocas
- ☐ Aumentar learning rate